

Academic Question Answering System Using RAG

Retrieval-Augmented Generation for Source-Cited Academic Answers

Prepared by	Vishesh Maurya
Department	Computer Science and Engineering
Institution	ABES Engineering College, Ghaziabad
Academic year	2026–2027

1. Abstract

The Academic Question Answering System Using Retrieval-Augmented Generation (RAG) answers academic questions using information retrieved from educational PDFs. It extracts text, divides it into chunks, converts chunks into vector embeddings, and stores them in a vector database. Relevant passages are retrieved for each question and supplied to a Large Language Model (LLM) to generate a context-grounded answer. PDF and page citations help students verify responses. The initial knowledge base focuses on AI and RAG resources.

2. Problem Statement

Manual searching across lengthy PDFs is time-consuming, keyword search can miss semantically relevant passages, and general AI chatbots may produce unsupported answers without sources. This project retrieves evidence from academic PDFs, generates answers from that evidence, and provides verifiable source and page references.

3. Aim and Objectives

Aim: Develop an AI-based academic question-answering system that provides document-grounded answers with verifiable citations.

- Extract PDF text while preserving page metadata.
- Chunk text, generate embeddings, and store vectors.
- Retrieve relevant passages for natural-language questions.
- Generate answers from retrieved context and display citations.
- Evaluate retrieval, answer correctness, faithfulness, and citation validity.

4. Scope

Initial scope: approximately 1–5 selectable-text PDFs about AI and RAG; text extraction, chunking, embeddings, vector retrieval, LLM answer generation, citations, and evaluation. Future extensions include OCR, more subjects, conversation history, highlighted citations, and web deployment.

5. Approach and Methodology

Stage	Process
1. Ingestion	Collect PDFs and extract text with PyMuPDF, retaining filename and original page number.
2. Preprocessing	Clean text and split into overlapping chunks. Starting setting: about 250 tokens per chunk and 60 tokens overlap; tune using tests.
3. Embeddings/indexing	Vectorize chunks with an embedding model and store vectors, text, and metadata in ChromaDB.
4. Retrieval	Embed the question and retrieve relevant chunks through similarity search.
5. Generation	Supply retrieved context to an LLM and instruct it to avoid unsupported claims.
6. Citations	Display answer with PDF and page references derived from metadata.

6. System Architecture

Document pipeline: PDFs → Text extraction → Cleaning/chunking → Source/page metadata → Embeddings → Vector database.

Question pipeline: User question → Query embedding → Similarity search → Relevant chunks → Context → LLM → Answer with citations → User interface.

Component	Responsibility
PDF processor	Extracts text and original page numbers.
Chunking/metadata	Creates passages and retains source details.
Embedding/vector store	Represents content as vectors and indexes it.
Retriever	Selects passages relevant to the question.
LLM/citation layer	Generates an evidence-based answer and source references.
Evaluation module	Checks retrieval, answer quality, faithfulness, and citations.

7. Technology Stack

Technology	Purpose
Python	Core language and pipeline orchestration.
PyMuPDF	PDF text and page extraction.
Sentence Transformers or configured model	Creates embeddings for semantic search.
ChromaDB	Vector storage and similarity retrieval.
Google Gemini API (if configured)	LLM-based answer generation.
LangChain / Streamlit (optional)	Text splitting/orchestration and UI, if used.
Git and virtual environment	Version control and dependency isolation.

Document actual installed dependencies and model names in the final submission. Optional tools are not assumed to be implemented.

8. Citation Mechanism and USP

Each chunk should retain document ID, source filename, original page number, chunk ID, and text. Displayed citations must come from retrieved metadata, not be invented by the LLM. Each cited passage must support the associated claim; citation validity should be tested separately from answer fluency.

Illustrative format: Answer followed by “rag_fundamentals.pdf — Page 4.” This is an example only; verify real citations against the dataset.

USP: Academic question answering with traceable, page-level evidence from user-provided study materials.

- Answers are grounded in retrieved academic content.
- Page-level traceability helps locate evidence in original PDFs.
- Semantic retrieval can match concepts when wording differs.
- The system can report when evidence is insufficient.
- Modular components can be improved independently.

Capability	Keyword search	General chatbot	Proposed RAG
Semantic retrieval	Limited in basic search	Usually	Yes
Uses supplied PDFs	If indexed	Not guaranteed	Core workflow
PDF/page citations	Depends on design	Not guaranteed	Core requirement
Evaluation on fixed corpus	Possible	Not inherent	Supported

9. Feasibility Analysis

Type	Assessment
Technical	Feasible as a small prototype using established Python libraries and selectable PDFs. Main challenges are retrieval relevance and citation validation.
Economic	Open-source libraries and local vector storage reduce costs; LLM APIs may have charges, quotas, and rate limits.
Operational	Users can index PDFs, ask questions, and inspect citations without understanding the underlying algorithms.
Schedule	Divide work into dataset preparation, indexing/retrieval, generation, and evaluation/documentation.

10. Requirements

Functional: Process PDFs, preserve page metadata, create embeddings, accept questions, retrieve passages, generate answers, display citations, handle unsupported questions, and support evaluation.

Non-functional: Accuracy, traceability, reasonable response time, modularity, scalability, secure API-key handling, error handling, and reproducible settings.

11. Dataset and Evaluation

The initial dataset consists of approximately 1–5 selectable-text PDFs about AI and RAG. Retain source filename, original page, chunk ID, text, and embedding for each chunk. Keep a separate evaluation set with questions, reference answer points, and expected source pages.

- Direct factual and conceptual questions.
- Paraphrased questions and questions requiring multiple passages.
- Out-of-scope questions not supported by the PDFs.
- Citation checks verifying that source and page support the answer.

Area	Suggested measures
Retrieval	Recall@K, Precision@K, Mean Reciprocal Rank where appropriate.
Answer quality	Correctness against reference answers, completeness, and faithfulness.
Citations	Citation validity, citation support, and page-reference accuracy.

12. Test Plan

ID	Scenario	Expected result
TC-01	Question covered by PDF	Relevant answer and citation.
TC-02	Paraphrased question	Relevant passage retrieved.
TC-03	Multi-passage question	Answer combines evidence and identifies sources.
TC-04	Unsupported question	System indicates insufficient evidence.
TC-05	Verify page reference	Page exists and supports the answer.
TC-06	Empty input/API failure	Handled without crashing.

Report actual test counts and metrics. Do not claim a numerical accuracy rate unless measured on the evaluation dataset.

13. Current Project Status

Area	Reported status
Evaluation	Latest reported run: success.
Automated tests	Latest reported run: passed.
Chunking	250-token target and 60-token overlap discussed.
Pipeline	Answer pipeline reported working; confirm current code and outputs.

Statuses are based on reported progress and should be confirmed against the current codebase. Passing tests does not establish perfect answer or citation accuracy.

14. Limitations and Future Enhancements

Limitations include small topic coverage, extraction issues with tables/equations, retrieval misses, possible unsupported LLM claims, API dependency, and limited generalizability of small tests. Future work may include OCR, hybrid search, reranking, clickable/highlighted citations, additional models, conversation history, document management, larger benchmarks, and web deployment.

15. Expected Outcomes and Conclusion

The expected result is a prototype that indexes academic PDFs, retrieves relevant passages, generates grounded answers, preserves page metadata, and presents verifiable citations. It demonstrates embeddings, vector databases, information retrieval, prompt design, and LLM integration. Further evaluation and citation validation are needed to establish reliability.

16. References

1. Lewis et al. (2020), “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.”

<https://arxiv.org/abs/2005.11401>

2. Reimers and Gurevych (2019), “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks.”

<https://arxiv.org/abs/1908.10084>

3. PyMuPDF Documentation

<https://pymupdf.readthedocs.io/>

4. Chroma Documentation

<https://docs.trychroma.com/>

5. Google Gemini API Documentation

<https://ai.google.dev/gemini-api/docs>

6. Sentence Transformers Documentation

<https://sbnet.net/>

Submission Checklist

- Add actual application screenshots and directory structure.
- Include sample answers and citations verified against the dataset.
- Add measured evaluation results and observed errors.
- Confirm library versions and exact embedding/LLM model names.

Prepared for academic project documentation • Computer Science and Engineering • 2026–2027